COS 521 Final Project

[redacted] January 15, 2019

We will start by introducing some notation we use throughout this paper. Unless otherwise specified, we assume all graphs to have n vertices. In a weighted graph, we refer to the *weighted degree* of a vertex as the sum of the weights of the edges incident to that vertex. We denote this quantity for a vertex $v \in V(G)$ by $d_w(v)$. Given an edge e we denote its weight by $w(e)$. All weights are assumed to be positive real numbers.

1 Introduction

In the fall of 2017, Aviad Rubinstein, Tselil Schramm, and Matt Weinberg published "Computing exact minimum cuts without knowing the graph." In it, they consider access to an oracle that when queried on any $X \subseteq V(G)$ returns the value of the cut $(X, V(G) \backslash X)$. The challenge then is to compute the global minimum cut in the graph with as few oracle queries as possible, without initial knowledge the graph. While stated in this way it may seem to be an unmotivated setup, it is a special case of *submodular minimization* which recently has raised plenty of interesting theoretical questions.

The authors consider the case of unweighted, undirected graphs. They note that it takes n^2 bits to represent the graph, and only $O(\log n)$ bits are revealed on each query, so it takes $\Omega(\frac{n^2}{\log n})$ queries to learn the entire graph exactly (from which of course a minimum cut could be computed). This raised the question of whether a minimum cut could be learned in a subquadratic number of queries, which would beat the number of queries needed to learn the graph. Their surprising answer was that this is possible: they demonstrate an algorithm to find the *exact* minimum cut in a graph with only $\tilde{O}(n)$ queries.[1]

The authors only considered unweighted, undirected graphs. In their paper they specifically mention as follow-up questions the study of whether a similar statement could be said for weighted or directed graphs, noting this to currently be an unsolved problem. We decided to tackle it for our project. The main component of this paper is a technical presentation of our results, but we have also included a brief discussion of our experience and lessons with attempting research at the end.

Our main result is an algorithm for computing an exact minimum cut with high probability in a weighted graph with bounded weights that uses subquadratic queries. Specifically, we will show that for a graph whose weights are in the set $\{1, 2, \ldots, n^k\}$ for $k < 0.5$ we can determine an exact minimum cut in $\tilde{O}(n^{1+2k}) = o(n^2)$ queries.

This paper will be self-contained, but also uses without proof some results from Rubinstein *et al.*, and moves quickly through details that are the same.

[1] In this paper we use $\tilde{O}(n)$ to denote $O(n \cdot \text{polylog}(n))$.

2 A Quick Review

We'll next present a high-level overview of the techniques Rubinstein *et al.* used to get to their result. Their ideas rely on the technique of *graph sampling* (also sometimes referred to as *graph skeletons* or *graph sparsification*). These are techniques that let us get from a starting graph G to a graph H with significantly less edges, such that all the cuts in H approximate the cuts in G well (we will later define this in a precise sense).

Once we have such a graph H, we can find every approximate minimum cut. Vertices that appear on the same side of every approximate minimum cut in H cannot be separated by a minimum cut in G, which means we can combine them to "supernodes." If there are not too many edges that participate in approximate minimum cuts in G, then we will see an algorithm to exactly learn all the edges between such supernodes (from which we can compute the exact minimum cut).

3 Bounding Edges Participating in Minimum Cuts

We start by introducing a fundamental structural difference between approximate minimum cuts in weighted graphs and approximate minimum cuts in unweighted graphs. This difference is in the number of edges that can participate in non-singleton approximate minimum cuts. The relevant result proved by Rubinstein *et al.* is stated below.

Lemma 2.6 (Rubinstein, Schramm, Weinberg). Let $G = (V, E)$ be an unweighted graph with minimum degree d and minimum cut value c. Let $\mathcal{C}$ be the set of all non-singleton approximate-minimum cuts in the graph, with cut value at most $c + \epsilon d$, for $\epsilon < 1$. Then $|\cup_{C \in \mathcal{C}} C|$ (the total number of edges that participate in cuts in $\mathcal{C}$) is $O(n)$.

Across different techniques in their paper, this structural result has to be used at the end to improve different approximate algorithms to exact algorithms. It's reasonable to conjecture that any sampling technique (which inherently only preserves *approximate* min-cuts) must in some way rely on a structural result of this form to end with an exact algorithm.

We wanted to see if there was any sort of structural result of this nature for weighted graphs, to improve approximation algorithms for weighted graphs to exact algorithms. Initially we hoped to find a similar subquadratic bound. Our first result here demonstrates that this is not possible in the most general weighted graph. However, it motivates consideration of what we call the *scaling factor*, which for a simple weighted graph G we define as

$$\gamma_G := \frac{e_{\max}}{e_{\min}}$$

where $e_{\max}$ is the maximum edge weight and $e_{\min}$ is the minimum edge weight.

Theorem 3.1. There exists a simple weighted graph G on n vertices with $\gamma_G = n$, such that the number of edges that participate in all non-singleton *exact* minimum cuts is $\Omega(n^2)$. (Note that stating this result for exact minimum cuts is strictly stronger than the target of approximate minimum cuts.)

Proof. For any $a \geq 1$, consider K_{2a}, the complete graph on $2a$ vertices which we will label v_1, v_2,

$\ldots$, v_{2a}. We will give the edge between v_i and v_j weight

$$w(v_i, v_j) := \begin{cases} 2a & \text{if } i \equiv j \pmod 2 \\ 1 & \text{else} \end{cases}.$$

Call a vertex v_i an *odd vertex* if i is odd, and an *even vertex* otherwise.

I claim the minimum cut value is a^2; this is certainly achievable when we place the odd vertices on the left of the cut and the even vertices on the right as there are a^2 edges crossing the cut which each have weight 1.

To see why this is optimal, assume we have some a minimum cut $(X, V(G)\backslash X)$. Say X has x odd vertices and y even vertices, so $V(G)\backslash X$ has $a-x$ odd vertices and $n-y$ even vertices. If $0 < x < a$, then the weight across the cut between odd vertices is

$$(2a)x(a-x)$$

which for $0 < x < a$ already is more than a^2 (an achievable cut value). So x is either 0 or a, similarly y is either 0 or a. If $x = y = 0$ and $x = y = a$ then we have an empty cut. Hence either $x = 0$ and $y = a$ or $x = a$ and $y = 0$; these correspond exactly to cuts that split even/odd vertices and have a value of a^2.

Hence, we have demonstrated that at least $a^2 = \Omega(n^2)$ edges participate in minimum cuts in this graph, proving the result. ■

We have hence demonstrated that Rubinstein *et al.*'s Lemma 2.6 cannot hold in general weighted graphs. However, our result motivates the consideration of graphs with bounded scaling factor. Our next result exactly demonstrates a bound on the number of edges that participate in approximate minimum cuts in a weighted graph which depends on this scaling factor. Our proof follows much of the structure of the proof in the unweighted case provided by Rubinstein *et al.* but most be modified for the weighted case. We also are able to fix minor typos in the original.

Theorem 3.2. For any constant $0 \leq k < 0.5$, let G be a simple weighted graph on n vertices with $\gamma_G \leq n^k$. Let c be the value of the minimum cut of G and let d be the minimum weighted degree of G. Assume $d > 0$ (if $d = 0$, we will find our minimum cut after our n oracle queries on single vertices at the start). Among all non-singleton cuts of G with cut value at most $c + \epsilon d$ for $\epsilon < 1$, at most $O(n^{1+2k})$ edges participate.

Proof. Let $e_{\min}$ denote the minimum edge weight in G and let $e_{\max}$ denote the maximum edge weight in G. Define

$$\alpha := \frac{1-\epsilon}{8e_{\max}}.$$

Let $\mathcal{C}$ denote the set of all non-singleton cuts of G with cut value at most $c + \epsilon d$ for $\epsilon < 1$. As noted by Rubinstein *et al.*, we have that any subset of $\mathcal{C}$ creates to a partition of the vertices of G (with two vertices in the same component if and only if they are on the same side of every cut in the subset). Hence pick a subset of $\mathcal{C}$ of minimum size, such that when adding any other cut no component of the vertex partition is split into two components both with size at least αd; call this subset X. By an inductive argument we then have $|X| \leq \frac{n}{\alpha d}$. The total weight of all the edges participating in cuts in X is at most

$$|X|(c + \epsilon d)$$

which means at most

$$\left(\frac{|X|}{e_{\min}}\right)(c+\epsilon d)$$

edges participate in cuts in X. That means that the number of vertices incident to at least αd edges in X is at most

$$\left(\frac{2|X|}{\alpha d e_{\min}}\right)(c+\epsilon d).$$

Call this set of vertices Y.

Consider now what happens when we add a new cut C to X. If it splits an existing component, one of the newly split components must have size $< \alpha d$; call this component B. I claim that every vertex in B is incident to at least αd edges in X; assume for the sake of contradiction there exists some vertex $b \in B$ for which this is not the case. We'll bound the weight of edges from b to other vertices on the same side of the cut C. All such vertices are either in B or are participate in a cut in X. The weight from b to other edges in B is at most $\alpha d e_{\max}$, and we assumed that b is incident to at most αd edges that participate in cuts in X. Hence the weight of edges from b to vertices on the same side of the cut is at most $e_{\max}(\alpha d + \alpha d)$, and there must be at least $\deg(b) - e_{\max}(2\alpha d)$ weight crossing the cut. If we moved b to the other side of C (which gives a valid cut because we assumed C to be non-singleton), the value of the cut would then have to decrease by at least

$$\deg(b) - 4e_{\max}\alpha d \geq d(1 - 4e_{\max}\alpha).$$

Because we can't beat the value of the minimum cut, we hence must have

$$d(1 - 4e_{\max}\alpha) \leq \epsilon d$$

which contradicts our definition of α. So every vertex in B is incident to at least αd edges in X. So the edges in X and edges incident with Y include every edge participating in a cut in $\mathcal{C}$. Our previous bounds hence give that the number of edges participating in cuts in $\mathcal{C}$ is at most

$$\left(\frac{|X|}{e_{\min}}\right)(c+\epsilon d) + d \cdot \left(\frac{2|X|}{\alpha d e_{\min}}\right)(c+\epsilon d) = \left(\frac{|X|}{e_{\min}}\right)\left(1+\frac{2}{\alpha}\right)(c+\epsilon d) \leq \left(\frac{|X|}{e_{\min}}\right)\left(1+\frac{2}{\alpha}\right)2c.$$

Using the definition of α, we can write $|X| \leq n/\alpha d = \frac{8ne_{\max}}{d(1-\epsilon)}$ which means the number of edges participating in cuts in $\mathcal{C}$ is at most

$$\begin{aligned}
\left(\frac{8ne_{\max}}{(1-\epsilon)e_{\min}}\right)\left(1+\frac{16e_{\max}}{1-\epsilon}\right)2c &= 16\gamma_G\left(\frac{n}{d(1-\epsilon)}\right)\left(1+\frac{16e_{\max}}{1-\epsilon}\right)\left(\frac{c}{d}\right) \\
&\leq \frac{16}{1-\epsilon}\gamma_G n\left(\frac{1}{d}+\frac{16}{1-\epsilon}\cdot\frac{e_{\max}}{d}\right) \\
&\leq \frac{16}{1-\epsilon}\gamma_G n\left(\frac{1}{d}+\frac{16}{1-\epsilon}\cdot\frac{e_{\max}}{d}\right) \\
&\leq O(\gamma_G^2 n).
\end{aligned}$$

We hence have the result. ■

4 Sampling Weighted Graphs

To use the result from the previous section, we want to be able to construct a graph with $o(n^2)$ edges that approximately preserves all cuts. Assume that we first do n oracle queries to determine the weighted degree of every vertex in G. Our first result is that we can efficiently run Karger's algorithm on G. For this section, we will always let c denote the value of the minimum cut in G.

Theorem 4.1. We can run k steps of Karger's algorithm on G with $O(k \log n)$ oracle queries. At the end of each step, we can report the total weight of the edges that remain in the graph.

We will prove this with a sequence of lemmas. These are inspired by work from Rubinstein *et al.* but require modification to work in the weighted case.

Lemma 4.1. Given a vertex $v \in V(G)$, with $O(\log n)$ oracle queries, we can choose a random edge incident to v such that edge e is chosen with probability $\frac{w(e)}{d_w(v)}$.

Proof. Given v, split the other $n-1$ vertices into two disjoint sets A and B with $|A| = \lfloor \frac{n-1}{2} \rfloor$ and $|B| = \lceil \frac{n-1}{2} \rceil$. After calling our weighted oracle on $\{v\}$, A, and $\{v\} \cup A$, we can determine the weight of the edges from v into A; similarly, after calling our weighted oracle on $\{v\}$, B, and $\{v\} \cup B$ we can determine the weight of the edges from v into B. We can then pick one of A or B with probability proportional to the weight entering it; after we have made this selection we can apply the exact same algorithm, continuing by splitting it in half. We hence use $O(\log n)$ oracle queries and end with an edge chosen with the claimed probability. ■

Lemma 4.2. If a graph has m edges, we can learn it exactly in $\tilde{O}(m)$ queries.

Proof. The same procedure as in Lemma 2.1 can be modified to learn neighbors while keeping track of the edges that we have learned so far (so we will not even "learn" the same edge twice). ■

Lemma 4.3. We can pick a random edge in G, with probability proportional to its weight, in $O(\log n)$ oracle queries.

Proof. We assume that we have already computed the weighted degree of each vertex, so our first step is to pick a vertex at random with probability proportional to its weighted degree (no oracle queries required). Then, we can pick an edge incident to this vertex, with probability proportional to its weighted degree. ■

This lemma is the key component to proving Theorem 2.1, as we will see.

Proof of Theorem 4.1. We will keep track of the state of our graph by considering buckets of vertices (so we will start with n buckets each with one vertex, and after k steps of Karger have a total of $n-k$ buckets). In the first step of Karger, we can pick a random edge with probability proportional to the weight

We will to demonstrate some properties of Karger's algorithm next: these are of the same nature to results we saw in COS 521 but applied to weighted graphs.

Lemma 4.4 (Karger). Say we run Karger's algorithm until the total weight of edges in G is cn. The probability the minimum cut survives is at least $1/4$.

Proof. While there is still cn weight remaining in the graph, the probability of contracting something from the minimum cut is at most $\frac{c}{cn} = \frac{1}{n}$. As we will certainly run no more than n iterations

of Karger, the minimum cut survives with probability

$$\left(1 - \frac{1}{n}\right)^n > \frac{1}{4}$$

as claimed. ■

A crucial idea to the algorithm suggested by Rubinstein *et al.* is to apply sampling to the graph after Karger, independently keeping each edge in the graph with probability p. The goal is to have all cuts well-preserved, and this is analyzed using a Chernoff bound. Their result specifically is stated below.

Lemma 2.9 (Rubinstein, Schramm, Weinberg). Let $G = (V, E)$ be a multigraph with minimum cut value c, and let $G' = (V, E')$ be the result of sampling each edge of E with probability $p \geq \min(\frac{40 \ln n}{\epsilon^2 c}, 1)$. Then with high probability, every cut of value k in G has value $(1 \pm \epsilon)pk$ in G'.

We apply this result to the weighted case. We initially hoped to be able to directly apply such sampling to weighted edges, but note that then a standard Chernoff bound on Bernoulli random variables has to be replaced with a Hoeffding bound on bounded random variables. That bound, when applied to n random variables, has a $-\frac{1}{n}$ term in the exponent on the RHS. We saw earlier $\Omega(n^2)$ edges could participate in minimum cuts in a weighted graph, which would mean for our graph to preserve cut values we wouldn't necessarily be getting to a graph with a subquadratic of edges. Other papers we found on graph sparsification, such as Karger's 1999 paper on "Random sampling in cut, flow, and network design problems," were also not able to avoid a factor of W (the total weight) in their algorithms, as opposed to the truly desirable $\log(W)$. The idea of edge-strength sampling (Benczúr and Karger, 2015) seemed promising to us as well, but we found approximating edge-strengths with oracle queries relied on Chernoff bounds that did not generalize to unbounded weights.

As before, we need an assumption of boundedness.

Theorem 4.2. Given an initial weighted graph G with edges taking weights in $\{1, 2, \ldots, n^k\}$, with $\tilde{O}(n^{1+k})$ oracle queries we can create a graph H with the same vertices and $O(n^{1+k})$ edges such that every cut of value v in G has a value in $[vp(1-\epsilon), vp(1+\epsilon)]$ in H.

Proof. We first will compute all the weighted degrees in $O(n)$ queries. We will guess values for c that increase exponentially from 1 to n^{1+k}; so consider setting $c' = 2^j$ for $0 \leq j \leq (1+k)\log_2(n)$. This adds at most a $\log(n)$ factor to our number of oracle queries (which won't be a problem). For some guess c' we will be within a factor of 2 of the true value c.

We will run Karger until the total weight of edges in the graph is $c'n$ (which can be done with $\tilde{O}(n)$ oracle queries by Theorem 4.1). A simple extension to Lemma 4.4 demonstrates that we have the minimum cut surviving with constant probability (assuming our guess for c' is within a constant factor of c).

At this point we have at most $c'n$ edges in G. We will apply Rubinstein's Lemma 2.9 to this graph; to do so we will imagine splitting each edge of weight x in our graph into x edges of weight 1. In this imaginary graph we would have at most $c'n^{1+k}$ unweighted edges; applying Rubinstein's Lemma 2.9 to this would tell us that if we sample each edge with probability $\min(\frac{40 \ln n}{\epsilon^2 c}, 1)$ we will get a graph that approximates all cuts well which has $\tilde{O}(n^{1+k})$ edges with high probability.

Of course, we can't independently sample each edge in our graph. However, we can note that in the same manner as an argument provided by Rubinstein *et al.*, this "imaginary" sampling would be

equivalent to simply sampling $\tilde{O}(n^{1+k})$ edges from G with probability proportional to their weight (and keeping an edge of weight 1 for each such sample). By Lemma 4.3, we can do this in $\tilde{O}(n^{1+k})$ oracle queries, demonstrating the result. ■

5 Putting Together the Algorithm

Once we have H as in Theorem 2.2, requiring $\tilde{O}(n^{1+k})$ queries, we can compute *all* the approximate minimum cuts in H. As observed by Rubinstein *et al.*, any two nodes that appear in every approximate minimum cut in H can be merged to a supernode. Now, we can use Theorem 3.2 to note that if our weights are $\{1, 2, \ldots, n^k\}$ for $k < 0.5$ we have at most $O(n^{1+2k})$ edges that go between such supernodes; we can learn all of them by Lemma 4.2 in $\tilde{O}(n^{1+2k})$ queries. From here we can of course compute the exact minimum cut, proving our main result.

6 A Parallel Oracle

In this section, we include an alternative setting that is related to the original problem. Assume that instead of our usual oracle, we have a parallel oracle, that can return the values of up to n cuts at the same time. We will provide a nice solution for the graph with weighted edges, without assuming bounds on their weights as before. The obvious solution is to learn all edges exactly, with $O(n)$ queries to the parallel oracle; we wanted to see if we could find a fundamentally different approach which uses $O(n)$ queries, to perhaps give structural insights. We found a solution using Karger-Stein (the extension to Karger's algorithm that we learned in COS 521).

Let's have n copies of the initial graph. Call them $G_1, ..., G_n$. Now, on each one, we will do a process similar to the Karger-Stein Algorithm. Our plan will be to run some version of that algorithm so that on each copy, we would need $O(n \log n)$ queries instead of n^2 as we would in the case of just applying Karger-Stein in the same form as seen in class. Let's consider one G_i and denote it by H as we will be focusing on its thread for now. We run the Karger Algorithm for $n - n/2 = n/2$ steps to get H_1 and we do it again to get a graph H_2. Then, we will do the same thing recursively with H_1, H_2. Let C be a minimum cut in H. Then, for each H_i the probability the cut survived is $\frac{n-2}{n}\frac{n-3}{n-1}...\frac{n/2-2}{n/2} = \frac{(n/2-1)(n/2-2)}{n(n-1)} \geq 1/4$. Let's denote by $P(k)$ the probability that an iteration of the modified Karger-Stein maintains the cut for the graph of size k.

Now, we will prove that $P(k) \geq \frac{t}{k \log k}$ for some small $t < 1$. This is clearly the case for $n = 1$ or n small since we can pick t to be very small. We will proceed by induction. Let's prove it for $P(n)$. We have that the probability of H_1's path succeeding is at least $1/4 \cdot P(n/2)$. Hence, the probability that both paths fail is at most $(1 - \frac{P(k)}{4})^2$, so the probability of success at the end of the entire algorithm is at least $1 - (1 - \frac{P(k)}{4})^2$. By the inductive hypothesis, this is at least $\frac{t}{2(n/2)(\log n-1)} - \frac{t^2}{n^2(\log n-1)^2} = \frac{t}{n(\log n-1)} - \frac{t^2}{n^2(\log n-1)^2} = \frac{t}{2(n/2)\log n} + \frac{t}{n \log n(\log n-1)} - \frac{t^2}{n^2(\log n-1)^2}$. Now, for n big enough, say $n > 20$ we have that $\frac{t}{n \log n(\log n-1)} > \frac{t^2}{n^2(\log n-1)^2}$. And for $n \leq 20$, just choose a $t < 1$ small enough so that it works.

Hence, we proved that running this version of Karger-Stein for G_1 gives us a $\frac{t}{n \log n}$ probability of success. Now comes the justification of choosing the n copies. Note that if we were do do the same process of G_1 for $(1/t)n(\log n)^2$ times, then we would get with probability at least $1-(1/n)$ we have that the min cut survives in at least one of them. Hence, we can do this algorithm we presented in n copies of G at a time, using the parallel oracle. Each sampling of an edge proportional to its weight takes $\log n$ queries with the old graph, so to sample an edge in each of the n copies we need $\log n$ queries of the parallel oracle. By the way we built the recursion, we have that it takes $n(\log n)^2$ old queries to finish each thread, so it takes $n(\log n)^2$ with the parallel oracle. After this, we keep in mind the best cut seen and repeat the process of making n copies of G for $(1/t)(\log n)^2$ times, to succeed with high probability.

Thus, we have an algorithm that uses $\tilde{O}(n)$ parallel oracle queries to find the exact min-cut of G with probability at least $1-(1/n)$.

7 Brief Notes on Other Problems

This section briefly documents some of the setups we tried that didn't lead to a new result, and why this was case. One of the setups we tackled was the problem for directed graphs; this seems to be a much harder problem than generalizing to weights and may require fundamentally new ideas. We first demonstrated with proof that the oracle cannot determine using finitely many queries, for any single pair of vertices u, v, if there is a directed edge from u, v. Furthermore, for each vertex u, we cannot even find a neighbor v, such that $u \to v$. To prove these claims, let's consider the graph G_1 (the directed cycle with $1 \to 2 \to \ldots \to n \to 1$) and G_2 (the directed cycle with $n \to n-1 \to \ldots \to 1 \to n$). Let f_1 denote the oracle for G_1 and let f_2 denote the oracle for G_2.

Claim. For every subset $S \subset [n]$, we have that $f_1(S) = f_2(S)$.

Proof. Let $C_1, ..., C_r$ the contiguous components of S (the components of the form $\{k, k+1, \ldots, j\}$ considered modulo n). It's easy to notice that in both G_1 and G_2, a contiguous component has one edge going to the outside and one from the outside going into it. Hence, we have that $f_1(S) = r = f_2(S)$. ■

This demonstrates that for fixed vertices u and v, no sequence of queries can tell us if $u \to v$ or $v \to u$. For the same graph, let's suppose that we can find a neighbor of 1 after some sequence of queries. The only possible neighbors could be 2 or n. Because we cannot distinguish between G_1 and G_2, we have this result as well.

This example consists of graphs that are isomorphic, but we also found such an example of two graphs G_1, G_2 whose cuts on every subset are the same that are not isomorphic (which can in fact be generalized to graphs with $2n+1$ vertices).[2]

The fact that it is impossible to learn the graph already doesn't inspire confidence in the existence of a query-efficient minimum cut algorithm, but there are other issues as well. Karger's algorithm does not work for directed graphs (the minimum cut in a directed graph could have many edges

[2]Proving that the cuts are the same is not difficult. We just consider what vertices the subset S has. It can have three types of vertices: the center, the inner circle and the outer circle. These three categories form a total of 8 cases and we can use an inductive argument.

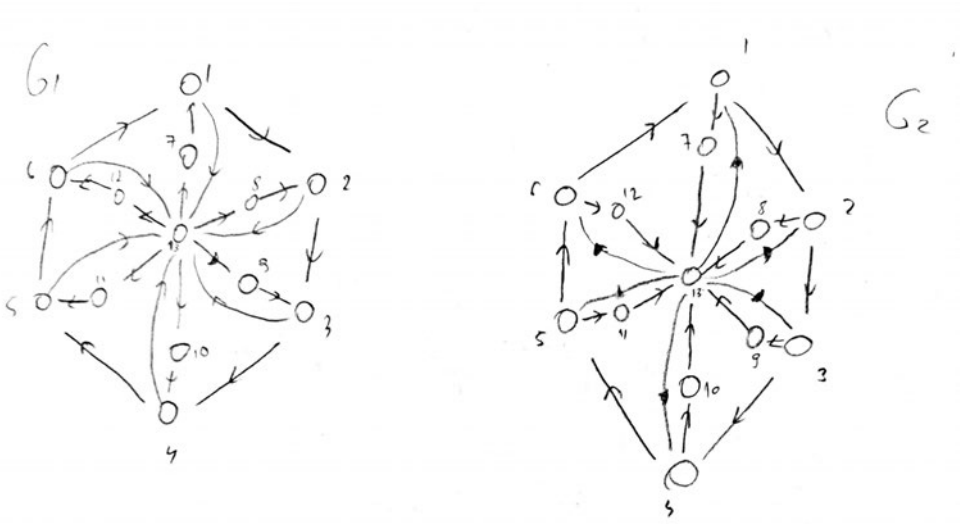


Figure 1: Graphs not isomorphic but with same cut values

that go backwards and are not counted). Furthermore, the literature of sampling/skeletons we surveyed pertains exclusively to undirected graphs.

We dedicated some time to perhaps finding a lower bound for oracle queries in the case where we have unbounded weights. Our idea was to consider the problem in the context of linear algebra: we need to solve for $\binom{n}{2}$ variables (the weight of each edge) that take non-negative values, and each oracle query gives us one equation. Hence if we only have $o(n^2)$ queries, the possible solutions over the real numbers will lie on a shifted linear subspace of dimension $\Omega(n^k)$ for some $k > 0$. For a fixed complete that has edge weights that are all positive, we would know one point in this space. The weighted case allows us the freedom of considering small shifts from this point; if any shift had a different minimum cut it would prove that the set of $o(n^2)$ queries considered could not determine the minimum cut. This is fundamentally different from the unweighted case, which is discretized in the sense that all variables must take 0/1 values; here the continuous nature means that small ϵ-shifts could be of use. In the time we had to consider this, we weren't able to get a full result leading to a lower bound, but are excited to continue work on this idea.

References

[1] András A Benczúr and David R Karger. Randomized approximation schemes for cuts and flows in capacitated graphs. *SIAM Journal on Computing*, 44(2):290–319, 2015.

[2] David R Karger. Global min-cuts in rnc, and other ramifications of a simple min-cut algorithm. In *SODA*, volume 93, pages 21–30, 1993.

[3] David R Karger. Random sampling in cut, flow, and network design problems. *Mathematics of Operations Research*, 24(2):383–413, 1999.

[4] Aviad Rubinstein, Tselil Schramm, and S Matthew Weinberg. Computing exact minimum cuts without knowing the graph. *arXiv preprint arXiv:1711.03165*, 2017.

A final note: In case it is of any interest to others (or perhaps our future selves), we thought it might be beneficial to include a brief non-technical summary of our experience working on this project. It was the first experience either of us had working on an open problem in theoretical computer science, and we feel that even in a few weeks we were able to learn a large amount about the research process and the important results in this area.

We firstly would like to say thank you to those who helped us along the way. In particular, the work of Pravesh Kothari and Christopher Musco in running COS 521 was what gave us the background and confidence to attack a problem of this nature. Our discussions with Christopher Musco as we started were interesting and helpful, giving us no shortage of questions to think about. We also would like to thank [redacted] for suggesting the problem.

We learned firsthand from this project about the amount of ideas that *don't* work out when tackling an open question. For every idea that we've included in this paper, there were 10 that simply didn't work out. This feeling is also present when working on problem-sets at times, but at a smaller scale than for a long-term project. Another new challenge was thinking about which questions are the "right" ones to ask (which of course is not required on problem sets).

We really enjoyed the ability to work as a team (and would suggest keeping this as an option in future iterations of COS 521). Sharing ideas, progress, and challenges was what made this project possible.

There is of course more research that can be done in this area. As the project progressed, we became increasingly interested in exploring lower bounds for general weighted or directed graphs; although we didn't have quite enough for a general theorem of this form we believe it to be entirely possible, and in the next few weeks hope to explore this idea further.